

Day 4 Notes SQL

Page 1 — SQL Basics and Data Retrieval

What is SQL?

SQL stands for **Structured Query Language**. It is used to communicate with relational databases such as MySQL, PostgreSQL, SQL Server, and Oracle.

Using SQL, we can:

- Retrieve data
- Insert new data
- Update existing data
- Delete data
- Filter records
- Sort records
- Perform calculations
- Group similar records

Assume we have an `employees` table with columns:

`employee_id, name, department, salary, city, age`

1. SELECT

`SELECT` is used to retrieve data from a table.

Basic syntax:

```
SELECT column_name FROM table_name;
```

Example:

```
SELECT name FROM employees;
```

This displays only the `name` column.

To retrieve multiple columns:

```
SELECT name, salary FROM employees;
```

To retrieve every column:

```
SELECT * FROM employees;
```

Here ***** means **all columns**.

2. DISTINCT

DISTINCT removes duplicate values from the result.

Example:

```
SELECT DISTINCT department FROM employees;
```

If many employees belong to IT, IT will appear only once.

It is useful when we want to know the unique values present in a column.

3. WHERE

WHERE is used to filter rows according to a condition.

Syntax:

```
SELECT column_name FROM table_name WHERE condition;
```

Example:

```
SELECT * FROM employees WHERE salary > 50000;
```

This returns employees whose salary is greater than 50,000.

Other examples:

```
SELECT * FROM employees WHERE city = 'Delhi';
```

```
SELECT * FROM employees WHERE age >= 25;
```

Common comparison operators:

- = Equal to
 - > Greater than
 - < Less than
 - >= Greater than or equal to
 - <= Less than or equal to
 - <> or != Not equal to
-

4. AND

AND is used when **all given conditions must be true**.

Example:

```
SELECT * FROM employees WHERE department = 'IT' AND salary > 50000;
```

The employee must:

- Belong to IT
- Have salary greater than 50,000

Both conditions must be satisfied.

5. OR

OR is used when **at least one condition must be true**.

Example:

```
SELECT * FROM employees WHERE city = 'Delhi' OR city = 'Mumbai';
```

Employees belonging to either Delhi or Mumbai will be returned.

6. NOT

NOT reverses a condition.

Example:

```
SELECT * FROM employees WHERE NOT city = 'Delhi';
```

This returns employees who are not from Delhi.

Another way:

```
SELECT * FROM employees WHERE city <> 'Delhi';
```

Page 2 — Filtering and Sorting Queries

7. BETWEEN

BETWEEN checks whether a value lies within a range.

Example:

```
SELECT * FROM employees WHERE salary BETWEEN 40000 AND 70000;
```

It includes both 40,000 and 70,000.

It can also be used with dates and other numeric values.

Example:

```
SELECT * FROM employees WHERE age BETWEEN 20 AND 30;
```

8. IN

IN is useful when we want to compare a column with multiple possible values.

Instead of writing:

```
city = 'Delhi' OR city = 'Mumbai' OR city = 'Noida'
```

We can write:

```
SELECT * FROM employees WHERE city IN ('Delhi', 'Mumbai', 'Noida');
```

It makes queries shorter and easier to understand.

9. NOT IN

NOT IN excludes multiple values.

Example:

```
SELECT * FROM employees WHERE city NOT IN ('Delhi', 'Mumbai');
```

This returns employees whose city is neither Delhi nor Mumbai.

10. LIKE

LIKE is used for pattern matching, mainly with strings.

Two important wildcard characters are:

% → represents zero or more characters.

_ → represents exactly one character.

Example:

```
SELECT * FROM employees WHERE name LIKE 'A%';
```

Returns names starting with A.

Examples:

```
LIKE '%a'
```

Names ending with a.

```
LIKE '%raj%'
```

Names containing raj.

```
LIKE 'A_'
```

Names containing exactly two characters and starting with A.

11. IS NULL

NULL represents a missing or unknown value.

We should not write:

```
department = NULL
```

Instead, use:

```
SELECT * FROM employees WHERE department IS NULL;
```

This returns rows where department has no value.

To find rows having a value:

```
SELECT * FROM employees WHERE department IS NOT NULL;
```

12. ORDER BY

ORDER BY is used to sort results.

Ascending order:

```
SELECT * FROM employees ORDER BY salary ASC;
```

ASC means ascending.

For numbers:

Small → Large

For text:

A → Z

Descending order:

```
SELECT * FROM employees ORDER BY salary DESC;
```

For numbers:

Large → Small

For text:

Z → A

If **ASC** or **DESC** is not specified, SQL generally uses ascending order.

13. Sorting Using Multiple Columns

We can sort using more than one column.

Example:

```
SELECT * FROM employees ORDER BY department ASC, salary DESC;
```

First, employees are arranged according to department.

Then, employees within the same department are arranged according to salary from highest to lowest.

14. LIMIT

LIMIT restricts the number of rows returned.

Example:

```
SELECT * FROM employees LIMIT 5;
```

Returns only the first five rows.

To find the five highest-paid employees:

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

This is a very common interview query.

Page 3 — Basic Modification and Aggregate Queries

15. INSERT

INSERT is used to add new records to a table.

Example:

```
INSERT INTO employees (employee_id, name, department, salary) VALUES  
(1, 'Rahul', 'IT', 60000);
```

This inserts one employee into the table.

Multiple records can also be inserted together.

16. UPDATE

UPDATE modifies existing records.

Example:

```
UPDATE employees SET salary = 70000 WHERE employee_id = 1;
```

This changes the salary of employee ID 1.

The **WHERE** condition is extremely important.

Without **WHERE**:

```
UPDATE employees SET salary = 70000;
```

The salary of **every employee** would become 70,000.

17. DELETE

DELETE removes rows from a table.

Example:

```
DELETE FROM employees WHERE employee_id = 1;
```

This deletes employee ID 1.

Without **WHERE**:

```
DELETE FROM employees;
```

Every row in the table gets deleted.

18. Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a result.

Important functions are:

COUNT

Counts records.

```
SELECT COUNT(*) FROM employees;
```

Returns the total number of employees.

SUM

Calculates total.

```
SELECT SUM(salary) FROM employees;
```

Returns the total salary of all employees.

AVG

Calculates average.

```
SELECT AVG(salary) FROM employees;
```

Returns the average employee salary.

MAX

Returns the maximum value.

```
SELECT MAX(salary) FROM employees;
```

Returns the highest salary.

MIN

Returns the minimum value.

```
SELECT MIN(salary) FROM employees;
```

Returns the lowest salary.

19. GROUP BY

GROUP BY combines rows containing the same value and is commonly used with aggregate functions.

Example:

```
SELECT department, COUNT(*) FROM employees GROUP BY department;
```

This tells us the number of employees in every department.

Another example:

```
SELECT department, AVG(salary) FROM employees GROUP BY department;
```

This gives the average salary of each department.

20. HAVING

HAVING filters the result created after GROUP BY.

Example:

```
SELECT department, AVG(salary) FROM employees GROUP BY department  
HAVING AVG(salary) > 50000;
```

First employees are grouped by department.

Then average salary is calculated.

Finally, only departments whose average salary is above 50,000 are returned.

Important difference:

WHERE → filters individual rows before grouping.

HAVING → filters groups after GROUP BY.

21. Aliases — AS

Aliases give temporary names to columns.

Example:

```
SELECT AVG(salary) AS average_salary FROM employees;
```

Instead of showing something like AVG(salary), the result column will be called average_salary.

Another example:

```
SELECT department, COUNT(*) AS employee_count FROM employees GROUP BY  
department;
```

Basic SQL Query Execution Order

For a query containing filtering and grouping, logically SQL processes it approximately as:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Remembering this order helps understand many SQL interview questions.

Most Important Basics for Interviews

You should be comfortable with:

SELECT
DISTINCT
WHERE
AND / OR / NOT
IN / NOT IN
BETWEEN
LIKE
IS NULL / IS NOT NULL
ORDER BY
LIMIT
INSERT
UPDATE
DELETE
COUNT
SUM
AVG
MIN
MAX
GROUP BY
HAVING
AS

These concepts form the foundation for moving to **joins, subqueries, CASE WHEN, CTEs, and window functions**.